

A GUIDE FOR MAHEEN

orbit: how it was *made* and how the code *works*

A plain-English walkthrough of your study app: what each part does, how the pieces talk to each other, and how to explain it if someone asks.

27 FILES · ABOUT 6,550 LINES OF CODE · ABOUT 6,600 LINES OF TEACHING COMMENTS

Contents

1. What orbit is
2. How it was made
3. The big picture: three layers
4. How a page appears on screen
5. Where your data lives
6. How the AI works
7. Feature by feature
8. Map of every file
9. Key ideas and words
10. If someone asks you...
11. Reading the code in VS Code

1. What orbit is

orbit is a personal study app that runs on your own laptop. You open it in a web browser at `http://127.0.0.1:3000`, but it isn't on the internet: a small program on your Mac (the *server*) sends the pages to your browser.

SECTION	WHAT IT DOES
home	Greeting, focus phase, an "orbit map" of upcoming deadlines, this week's tasks, countdowns, music, recent notes. Cards can be moved around.
calendar	Month view of every event, colour-coded by course. Export to Google or Apple Calendar.
syllabus	Drop in a syllabus (PDF, Word, photo...) and every date lands on the calendar automatically, plus study sessions before exams.
notes	A folder per course. Turn lectures into study notes. A cheat sheet and practice test are made a few days before each exam.
ask	A chat assistant that knows your calendar, notes and degree. You choose which AI model answers.
focus	A pomodoro timer whose "phase" gets more intense as an exam gets closer (drift → rising → gravity → eclipse → liftoff).
degree	Year folders of courses, credits left to graduate, average and GPA, and an automatic check of your program requirements.
careers	Search internships and jobs (Canada first), tailor resumes and cover letters with AI, and track applications.
settings	Name, theme, automation switches, AI model, Spotify, courses, backup and restore.

2. How it was made

orbit was built in a conversation between you and Kiro, an AI coding assistant. You described what you wanted ("a bot like my Notion with a built-in calendar..."), Kiro wrote the code and tested it, and you gave feedback ("too AI-looking", "make it Canada-based", "the add box doesn't let me type"). Each round became a saved version, called a *commit*. There are 15 commits in total:

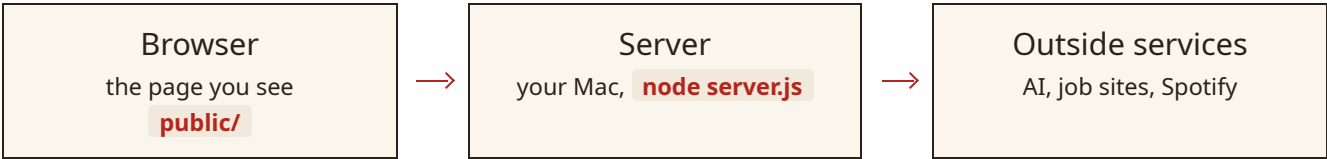
1. **Version 1:** calendar, syllabus import, chat, notes, focus modes, Spotify.
2. **Kiro as the AI:** the app talks to your Kiro subscription.
3. **Spotify setup help** and clearer error messages.
4. **The redesign as "orbit":** a paper and espresso look, serif headlines, orbit theme.
5. **Careers:** job finder, resume tailoring, application tracker.
6. **Canada first:** Canadian job lists, provinces, co-op terms, work permits.
7. **Any file format:** Word, PowerPoint, Excel, photos and scanned PDFs.
8. **Model picker:** choose Kiro, Claude, ChatGPT or Gemini models.
9. **Bug fix:** the quick-add box swallowed every key press.
10. **Degree planner.**
11. **Whole-web job search:** Google Jobs, Adzuna, Jooble.

12. **Streams and course folders:** the degree planner asks which stream you're in; notes are sorted into course folders; exam prep is automatic.
13. **More reliable job search:** retries and a "test my sources" button.
14. **Movable home cards** and themed dropdowns everywhere.
15. **Model menu fix:** the menu no longer gets cut off.

Choices made along the way, and why

- **No frameworks or libraries.** The building environment couldn't download packages from npm (the usual JavaScript package store), so orbit uses only what's built into Node.js and the browser. One upside: you never run `npm install`, and nothing can break because a package was updated. The only outside code is two libraries loaded from the web on demand: pdf.js (reads PDFs) and Tesseract (reads text in photos).
- **Local-first.** Your data stays in your browser. There's no account, database or cloud.
- **AI is optional.** Every AI feature has an offline fallback (for example, a date-finding parser for syllabi) so the app still works without a key.
- **Tested at every step.** Each feature was checked in a real (headless) Chrome browser. Services the sandbox couldn't reach (the AIs, job sites, Spotify) were replaced with stand-ins that reply the way the real ones do.

3. The big picture: three layers



1. The browser (front end)

Everything in the `public/` folder. `index.html` is the only HTML page, `styles.css` is the look, and `public/js/*.js` are the pages and logic. This is what you click on.

2. The server (back end)

`server.js` plus a few helper files. It sends the page files to the browser and offers a small *API* (web addresses starting with `/api/`) for anything that needs a secret key or has to read other websites.

3. Outside services

AI models (Kiro, Anthropic, OpenAI, Gemini), job sources (Google Jobs via SerpApi, Adzuna, Jooble, community GitHub lists, company job boards), university calendar pages, and Spotify.

Why have a server at all? Two reasons. (1) API keys are passwords. If they were in browser code, anyone could read them, so they stay in `.env` on the server. (2) Browsers block a page from downloading other websites (a security rule called CORS), so the server fetches job listings and calendar pages for it.

The server's API

ADDRESS	WHAT IT DOES	CODE
GET <code>/api/status</code>	Which AI providers, Spotify and job sources are set up	<code>server.js</code>
GET <code>/api/models</code>	The list of AI models for the model picker	<code>ai-providers.js</code>
POST <code>/api/ai</code>	Send a prompt to the chosen model and get its reply	<code>ai-providers.js</code>
POST <code>/api/jobs/search</code>	Search all job sources with your filters	<code>jobs.js</code> , <code>websearch.js</code>
GET <code>/api/jobs/test</code>	The "test my sources" button	<code>websearch.js</code>
GET <code>/api/job-text</code>	Read a job posting from a link	<code>jobs.js</code>
GET <code>/api/page-text</code>	Read a university calendar page from a link	<code>pages.js</code>
POST <code>/api/convert</code>	Convert old <code>.doc</code> files and HEIC photos using macOS tools	<code>server.js</code>

4. How a page appears on screen

orbit is a *single-page app*: the browser loads one HTML page once, and JavaScript swaps what's inside the main area.

1. You click **calendar** in the sidebar. The link is `#/calendar`, so only the part after `#` changes and the page doesn't reload.
2. The browser fires a `hashchange` event. `app.js` (the *router*) reads "calendar" and finds `calendar.js` in its route table.
3. It calls that file's `render(el)` function, where `el` is the empty `<main id="view">` box.
4. `render` builds the whole page as one long string of HTML (a *template literal*) from your saved data and puts it in with `el.innerHTML = ...`.
5. It then *wires* the page: it attaches functions to buttons and inputs (for example, "when this is clicked, add an event").
6. When you change something, the code saves it with `store.update(...)` and calls `draw()` again to rebuild the page.

Every page file follows this same pattern: **read data** → **build HTML** → **wire handlers** → **save and redraw**. Once you understand one page, you understand them all.

```
// the pattern, simplified
export function render(el) {
  const draw = () => {
    const data = store.get();           // 1. read
    el.innerHTML = `<button id="add">add</button>`; // 2. build
    el.querySelector("#add").onclick = () =>      // 3. wire
      store.update(s => s.events.push({...})); // 4. save (this triggers a redraw)
  };
  draw();
  return store.subscribe(draw);          // redraw whenever data changes
}
```

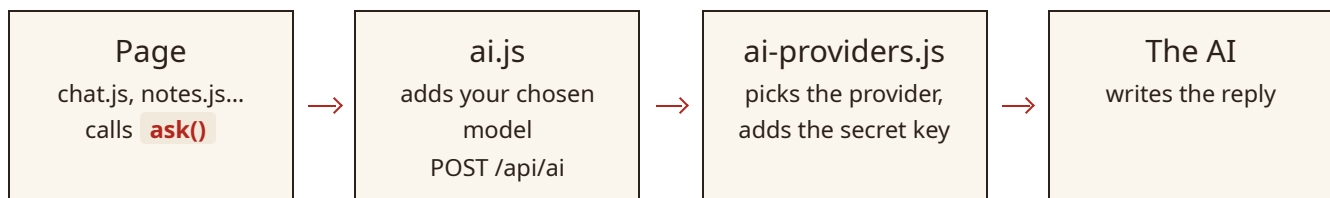
5. Where your data lives

All your courses, events, notes, chats, degree plan and career info sit in one JavaScript object called `state`, managed by `store.js`. It's saved in the browser's **localStorage** (a small storage area each website gets) as text in JSON format, under the key `studynos:v1`.

- `store.get()` reads the data.
- `store.update(fn)` changes it, saves it, and tells every page that's listening.
- `store.subscribe(fn)` means "call me whenever data changes". This pattern is called *publish/subscribe*.

Because data lives in the browser, a different browser (Safari vs Chrome) won't see it. Use **settings** → **export backup** to save a copy. Your keys live separately, in the `.env` file on your Mac.

6. How the AI works



- **A prompt has two parts.** The *system* message gives instructions ("you are a study assistant..."), and the *messages* are the conversation. Each feature writes its own instructions. For example, the resume writer is told never to invent experience, and the syllabus reader must return JSON.
- **JSON from the AI.** When the app needs structured data (events from a syllabus, requirements from a calendar page), the prompt asks for JSON. `extractJSON()` in `util.js` then digs it out of the reply, even if the AI wrapped it in extra text.
- **Context.** The chat attaches a summary of your data to every question (today's date, upcoming events, courses, note titles, focus phase, job applications, degree progress). That's how it can answer "what's due this week?" about *your* week.
- **Actions.** To add an event, the chat AI writes a special line like `<<ADD_EVENT {...}>>` . `applyActions()` finds those lines, adds the events, and hides them from the reply you see.
- **Choosing a model.** The picker lists models from every provider you've connected. Kiro is run as a command-line program (`kiro-cli`) in *headless* mode. Kiro can't switch models per question, so orbit writes a small text-only "agent" settings file for each model you pick (with no tools allowed, so it can't touch your files).
- **Offline fallbacks.** With no AI connected, the syllabus uses a date parser, notes use keyword scoring to pick the important sentences, and the chat answers schedule questions with simple rules.

7. Feature by feature

Syllabus import `SYLLABUS.JS` · `FILEREAD.JS`

1. **Reading the file:** `fileread.js` turns any file into plain text:

- **Word, PowerPoint and Excel** files are actually zip folders of XML, so orbit unzips them itself and reads the XML.
- **PDFs** are read with `pdf.js`.
- **Photos and scanned PDFs** are read with OCR (Tesseract "looks" at the image and guesses the letters).

2. **Finding events:** with AI, the text goes to the AI, which returns a JSON list of events. Without AI, *regular expressions* (text patterns) find dates like "Oct 14", "10/14" or "14th November" on each

line. The rest of the line becomes the title, and words like "midterm" or "quiz" decide the type.

3. **Saving:** the events are saved, and `syncStudyPlans()` in `focus.js` adds study sessions 10, 7, 5, 3, 2 and 1 days before each exam.

Calendar and home CALENDAR.JS · DASHBOARD.JS

- **The month grid** is always 42 squares (6 weeks × 7 days), starting from the Sunday before the 1st of the month.
- **Quick add** ("bio quiz fri 2pm") uses the same date and time patterns as the syllabus reader.
- **The orbit map** on the home page is an SVG drawing. Today is the sun, and deadlines are planets on rings for 7, 14 and 30 days away.
- **Movable home cards** use the browser's drag-and-drop events (`dragstart`, `dragover`, `drop`). Your layout is saved in settings.

Focus and Spotify FOCUS.JS · SPOTIFY.JS

- **The phase** comes from the number of days until your next exam: more than 14 → drift, 14 or fewer → rising, 7 or fewer → gravity, 3 or fewer → eclipse, exam day → liftoff. Each phase sets the timer lengths, the accent colour and the playlist.
- **The timer** saves the exact moment it should end (`endAt`) and works out "time left = endAt - now" every half second. That stays accurate even when the browser slows timers down in background tabs.
- **Spotify** works in two ways:
 - The embedded player needs no login.
 - "Connect" uses *OAuth with PKCE*, Spotify's secure login. You agree on Spotify's site, Spotify sends back a one-time code, and orbit trades the code for tokens (temporary passes).

Notes and exam prep NOTES.JS · EXAMPREP.JS

- **Folders:** each note has a `courseId`, and folders are just notes grouped by course.
- **Formatting:** notes are written in *Markdown* (`**bold**`, `## heading`, `==highlight==`). `md()` in `util.js` turns it into HTML.
- **Exam prep:** `autoPrep()` runs when the app opens and every hour after. For each exam within the chosen number of days, it sends that course's notes to the AI and saves two new notes: a cheat sheet and a practice test.

Degree planner DEGREE.JS · DEGREE-ENGINE.JS

- **degree-engine.js does the maths and has no page code**, so it can be tested on its own.
- **Course patterns:** `codeMatches('PSY3XX', 'PSY312H1')` is true, because X means any digit.
- **Matching courses to requirements:** specific courses are matched first, then narrow lists, then broad ones. That way one course isn't used up by a broad rule ("any PSY") when a specific rule needed it. A course counts once, unless a requirement allows overlap.

- **Average and GPA** are *weighted* by credits, so a full-year course counts twice as much as a half course.
- **Reading requirements:** the server fetches your program's calendar page (pages.js), and the AI turns it into a checklist. If the program has several **streams** (like CMP1 at UTM), the AI returns each one and the page asks you to pick yours.

Careers CAREERS.JS · JOBS.JS · WEBSEARCH.JS

- **Typed searches:** "summer 2027 co-op in toronto or remote" becomes filters. Patterns pick out the season and year, city names, "remote", the field and the job type.
- **Gathering jobs (server):** jobs.js collects jobs from community lists, company job boards (Greenhouse, Lever, Ashby) and whole-web search (websearch.js : Google Jobs, Adzuna, Jooble). Every source is converted to the same shape, filtered, and de-duplicated.
- **Fit score:** points for matching your target roles, skills, resume words and preferred location, plus freshness. The AI can then re-rank the top 30.
- **Resume tailoring:** the prompt follows strict honesty rules (reorder and reword, but never invent) and Canadian conventions (spelling, no photo, age or SIN). "Save as PDF" opens a print-ready page.

8. Map of every file

FILE	JOB
SERVER (RUNS ON YOUR MAC)	
server.js	Starts the web server, loads <code>.env</code> , serves files, handles every <code>/api/</code> address
ai-providers.js	Talks to Kiro, Anthropic, OpenAI and Gemini; lists models
jobs.js	Job lists, company boards, filtering, de-duplication, reading job postings
websearch.js	Google Jobs, Adzuna and Jooble searches, with caching and retries
pages.js	Reads the text of a university calendar page
kiro-agent/	Settings for the text-only Kiro agent
BROWSER: THE SHELL	
index.html	The one HTML page: sidebar, main area, toasts
styles.css	The whole look: theme colours, layout, every page's styles
app.js	Router; theme and focus colours; mini timer; startup
store.js	Your saved data plus helpers to change it
util.js	Dates, safe HTML (<code>esc</code>), ids, Markdown (<code>md</code>), JSON extraction
ui.js	Toasts, pop-up dialogs, spinner
ai.js	The browser side of the AI: <code>ask()</code> , the chosen model
modelpicker.js	The model picker button and menu
cselect.js	Turns every dropdown into a themed one
fileread.js	Reads PDF, Office, photo and other files into text
BROWSER: THE PAGES	
dashboard.js	home
calendar.js	calendar and the event editor
syllabus.js	syllabus import plus the offline date parser
notes.js · examprep.js	notebook, folders, cheat sheets and practice tests
chat.js	ask
focus.js · spotify.js	focus phases, timer, study plans, music

degree.js · degree-engine.js	degree planner (page · maths)
careers.js	careers: find, tailor, tracker, profile
settings.js	settings

9. Key ideas and words

WORD	MEANING IN PLAIN ENGLISH
Front end / back end	The part in the browser that you see / the part on the server you don't.
Server	A program that waits for requests and sends back answers. Here it's <code>node server.js</code> .
API	A set of web addresses programs use to talk to each other, usually sending JSON.
JSON	A text format for data, like <code>{"title": "Midterm", "date": "2026-10-14"}</code> .
API key / .env	A password for a service, kept in the <code>.env</code> file on your Mac and never sent to the browser.
DOM	The browser's live tree of everything on the page. JavaScript changes it to update what you see.
Event handler	A function that runs when something happens (a click, a key press, a file drop).
Template literal	A string in backticks that can insert values: <code>`Hi \${name}`</code> .
Arrow function	A short way to write a function: <code>(x) => x * 2</code> .
async / await	Lets code wait for slow things (network, files) without freezing the app.
Promise	A "result that arrives later". You <code>await</code> it.
Regex	A pattern for finding text, e.g. <code>/\d{4}-\d{2}-\d{2}/</code> finds dates like 2026-10-14.
localStorage	Small storage that each website gets in your browser. It survives reloads.
Single-page app	One HTML page whose content JavaScript swaps, instead of loading new pages.
Router	The code that decides which page to show for an address (app.js).
Module (import/export)	Each file shares some functions (<code>export</code>) and uses others' (<code>import</code>).
Cache	Saving a result for a while so you don't have to fetch it again (job searches are kept for 3 hours).
OCR	Software that reads letters out of a picture.
OAuth / token	A "log in with..." system. A token is a temporary pass you get after logging in.
Commit / git	Git saves snapshots of the code over time; each snapshot is a commit.
Escaping (XSS)	<code>esc()</code> turns <code><</code> into <code>&lt;</code> so text can never run as code inside the page.

10. If someone asks you...

"What did you build it with?"

Plain JavaScript, with no frameworks. Node.js runs a small server on my laptop, and the front end is vanilla JS modules, HTML and CSS. It has zero npm dependencies.

"Did you code it yourself?"

I designed it and directed it feature by feature, and I built it with an AI coding assistant (Kiro), which wrote the code and tested it in a browser. I've gone through it with line-by-line comments, so I can explain how each part works.

"Where's the data stored?"

In the browser's localStorage, so it's local-first: no account and no cloud. API keys stay in a `.env` file on the server and never reach the browser.

"How does the syllabus import work?"

It reads the file into text in the browser (it unzips Word files, uses pdf.js for PDFs and OCR for photos). Then the AI pulls out the dates as JSON, or, without AI, regex patterns find dates line by line. The events go on the calendar, and study sessions are scheduled before each exam.

"How does the AI know my schedule?"

Every question is sent with a short summary of my data as part of the instructions, so the AI is grounded in my calendar and notes rather than guessing.

"How does it find jobs?"

The server searches several sources at once: community internship lists, company job boards, Google Jobs, Adzuna and Jooble. It converts them to one format, filters by what I typed (term, city, remote, field, work permit), removes duplicates and ranks the results by fit with my profile.

"How does the degree check work?"

The calendar page is read by the server, and the AI turns it into a structured checklist. Then my own matching code assigns courses to requirements, most specific first, so a course isn't double-counted. It shows what's met, in progress, planned or missing.

"What was hard?"

Good examples: reading Word, Excel and PowerPoint files without any libraries (they're zip files of XML, so orbit has its own small unzipper), keeping the timer accurate in background tabs, matching courses to requirements without double-counting, and making every feature still work with no AI connected.

11. Reading the code in VS Code

- **Open the project:** in VS Code choose File → Open Folder and pick `~/Documents/study-os`.
- **Where to start:** read in this order: `index.html` → `app.js` → `store.js` → `util.js` → `dashboard.js`. After that, any page file will make sense.
- **How the comments are laid out:** every file starts with a block explaining its job. Every function has a short note on its inputs and what it returns, and most lines have a comment above them. JavaScript ideas are explained the first time they appear in each file.
- **Following the code:** **Cmd + click** a function name to jump to where it's defined. **Cmd + Shift + F** searches every file (try searching `store.update`).
- **Reading without comments:** **Cmd + K** then **Cmd + O** folds every section; **Cmd + K** then **Cmd + J** unfolds them again.
- **Experimenting:** change a colour in `styles.css` (try `--accent`), save, and reload the page to see what it controls. If something breaks, `git checkout` puts everything back.

The comments were added without changing a single line of code. An automatic check stripped the comments from every file and confirmed it was identical to the version before.